

In-Class Lab 01 - Create a PDO Connection

Course: WEB-250

Time: About 15-20 minutes

Grading: Practice only; nothing to submit

Continue in: [web250/salamanders](#)

Before starting: Complete the shared setup in [README-START-HERE.md](#).

Goal

Connect PHP to [wnc_salamanders](#) using a PDO object managed by a small [Database](#) class. This is the connection pattern used in [asgn07](#). Today, stop when the connection works; Lab 02 will run queries, and Lab 03 will handle failures.

You already know how to create a database. Our question now is: **How does a PHP program open a connection to that database?** PDO supplies the PHP API; its MySQL driver talks to the local MySQL/MariaDB server. [PDO introduction](#).

Step 1 - Create the local credentials file (about 3 minutes)

Copy [private/db_credentials.php.example](#) and name the copy [private/db_credentials.php](#). Leave the example unchanged. In the copy, replace the placeholder username/password and confirm the database port:

```
<?php
const DB_HOST = '127.0.0.1';
const DB_PORT = 3306;
const DB_NAME = 'wnc_salamanders';
const DB_USER = 'your_local_database_user';
const DB_PASS = 'your_local_database_password';
```

Use the local database credentials you already use in class. [3306](#) is an example, not a required port. MAMP often uses **8889 for MySQL** and **8888 for Apache**; other installations differ. Use the MySQL port here and the Apache port in the browser URL. Do not change working server settings just to match the example. [MAMP ports](#).

[127.0.0.1](#) selects a local TCP connection; it keeps the database port relevant rather than relying on the Unix socket behavior of [localhost](#). [PDO MySQL DSN](#).

Check that [.gitignore](#) contains [/private/db_credentials.php](#). We are keeping this configuration approach for now, not introducing a [.env](#) file.

Step 2 - Build the connection class (about 7 minutes)

Replace **all** of [private/classes/Database.php](#) with:

```
<?php

class Database
{
    // Store a PDO object, or null before the first connection.
    private static ?PDO $connection = null;

    public static function connect(): PDO
    {
        // Reuse this request's connection instead of opening another one.
        if (self::$connection instanceof PDO) {
            return self::$connection;
        }

        // The DSN tells PDO which driver, server, and database to use.
        $dsn = 'mysql:host=' . DB_HOST
            . ';port=' . DB_PORT
            . ';dbname=' . DB_NAME
            . ';charset=utf8mb4';

        self::$connection = new PDO(
            $dsn,
            DB_USER,
            DB_PASS,
            [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]
        );

        return self::$connection;
    }
}
```

Read the code in this order:

Code	Meaning in this application
<code>private static ?PDO \$connection = null;</code>	One class property stores the connection. The <code>?</code> allows <code>null</code> before there is a PDO object. Other code cannot directly change this private property.
<code>public static function connect(): PDO</code>	The page can call this method using the class name. A successful call returns a PDO object.
<code>self::\$connection</code>	Access the static property of this class, not a property on a <code>Database</code> instance.
<code>instanceof PDO</code>	Check whether that property already contains a PDO object.

Code	Meaning in this application
<code>\$dsn</code>	The Data Source Name: driver, host, database port, database name, and character set. The dots concatenate the pieces.
<code>new PDO(...)</code>	Create the connection object using the DSN, username, password, and options.

A second call to `Database::connect()` in the **same request** reuses the stored object. This is not a connection shared permanently across all visitors. You do not create `new Database()` because `connect()` is static. [Static methods and properties](#).

The options match asgn07. `ERRMODE_EXCEPTION` makes PDO report query failures as exceptions; we will catch those in Lab 03. `FETCH_ASSOC` sets associative arrays as the default for row fetching, and `ATTR_EMULATE_PREPARES => false` requests native prepared statements for later work. You do not need to use row fetching or prepared statements today. [PDO attributes](#).

Checkpoint: Point to the one line that actually starts a new connection. It is the assignment containing `new PDO(...)`, not the class declaration or `require_once`.

Step 3 - Call the method from the browser page (about 4 minutes)

Replace **all** of `public/index.php` with:

```
<?php
require_once __DIR__ . '/../private/initialize.php';

$database = Database::connect();
?>
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Salamanders - Connection Check</title>
</head>
<body>
  <h1>WNC Salamanders</h1>
  <p>PDO connection successful.</p>
</body>
</html>
```

`initialize.php` loads the definitions. `$database = Database::connect();` then calls the method and saves the returned PDO object in a variable. Merely including `Database.php` does not open the connection.

Refresh the **same MAMP/XAMPP URL** that displayed the starter page. Keep both Apache and the database server running.

Expected browser text:

```
WNC Salamanders
PDO connection successful.
```

That message is reached only after `connect()` returns successfully. It does not prove that every table or query is correct; those checks come next.

We have intentionally not added `try/catch` yet. Keep this intermediate version local. On a connection problem, PHP may show an uncaught exception or a generic error, depending on local settings. Check that MySQL is running, then check the database name, database port, and credentials with the instructor.

Do not deliberately break the connection until Lab 03.

Step 4 - Watch connection reuse (about 2 minutes)

Immediately after `$database = Database::connect();`, temporarily add:

```
$another_connection = Database::connect();
var_dump($database === $another_connection);
```

Refresh. Expect `bool(true)` in addition to the page. For objects, `===` tests whether the variables refer to the same instance; here, both calls returned the stored PDO object. [Object comparison](#).

Remove those two experiment lines. Keep the working connection class and page for Lab 02.

Explain it to a partner (about 2 minutes)

What is the difference between `Database` and `PDO`? Explain why the credentials file contains a database port even though the browser already uses a port.

Ready for the next lab: The browser reaches the success message, the credentials file stays local, and `Database.php` contains the reusable connection method.

Connection to asgn07

The `Database` class can stay the same in the bird application. Its separate credentials file selects `wnc_birds` instead of `wnc_salamanders`. Keep both projects; do not rename the salamander database or overwrite the bird project.